

Lernskript Programmierung 1 - Java-Grundlagen

Stoff: Foliensätze 00 bis 04 (Kursmodalitäten, Einführung, Datentypen/Operatoren/Verzweigungen, Schleifen, Methoden)

Ziel: Du verstehst die Konzepte **und** kannst sie selbst in Java programmieren.

Java-Version im Kurs: Java 21 · **Editor:** Visual Studio Code

So arbeitest du mit diesem Skript

1. Kapitel lesen und jedes Codebeispiel **selbst abtippen** (nicht kopieren!). Programmieren lernt man mit den Fingern.
2. Jedes Kapitel endet mit einem  **Selbsttest**. Beantworte ihn erst ohne Hilfe, dann klapp die Lösung auf.
3. Die ✂ **Übungen** zuerst allein lösen. Getestete Musterlösungen stehen im Anhang.
4. Wenn etwas nicht kompiliert, lies die Fehlermeldung. Kapitel 10 erklärt die häufigsten.

Inhaltsverzeichnis

0. Organisatorisches zum Kurs
 1. Grundlagen: Wie Computer „denken“
 2. Computational Thinking: Probleme lösen, bevor man programmiert
 3. Java, JVM & dein erstes Programm
 4. Datentypen, Variablen, Konstanten & Casting
 5. Ein- und Ausgabe auf der Konsole
 6. Operatoren & Ausdrücke
 7. Verzweigungen: if, else, switch
 8. Schleifen: while, for, do-while, break, continue
 9. Methoden
 10. Fehler lesen & Debugging
 11. Übungsaufgaben mit Lösungen
 12. Spickzettel (Cheat Sheet)
-

0. Organisatorisches zum Kurs

Punkt	Inhalt
Umfang	4 SWS, 8 ECTS → ca. 200 h Arbeitsaufwand, davon nur 48 h im Hörsaal → ~152 h Selbststudium
Anwesenheit	Pflicht
Werkzeuge	eigener Laptop, VS Code , Git , GitHub -Account (mit HCW-Mailadresse!), Java 21
Übungen	über GitHub Classroom bereitgestellt und abgegeben (Details auf Moodle)

Benotung (100 Punkte, bestanden ab 60):

Teilleistung	Punkte
Übungen	20
1. praktischer Test	20
2. praktischer Test	20
Theoretische Prüfung	20
Gruppenprojekt	20
Präsentation von Übungsaufgaben	bis zu 5 (Bonus)
Escape-Room-Teilnahme	5 (Bonus)

⚠ Teilleistungen können **nicht wiederholt** werden (immanente Leistungsüberprüfung). Unter 60 Punkten folgt ein Zweitantritt bzw. ein kommissioneller Antritt (praktisch + theoretisch).

Semesterplan: 1 Einführung → 2 Datentypen/Operatoren/Verzweigungen → 3 Schleifen → 4 Methoden → 5 Arrays → 6 OOP-Einführung → 7 OOP fortgeschritten → 8 Vererbung → 9 Abstrakte Klassen & Interfaces → 10 Collections & Exceptions → 11 Theorieprüfung → JavaFX + Git-Projekt → Escape Room → Projektdemos.

💡 **Warum das wichtig ist:** Alles ab Block 5 baut direkt auf den Blöcken 1 bis 4 auf, also auf diesem Skript. Wer Variablen, `if`, Schleifen und Methoden sicher beherrscht, hat den Rest viel leichter.

„Learning to make things requires to make things.“ – Thomas Dullien

Übersetzt: **Üben, üben, üben.** Lesen allein reicht beim Programmieren nicht.

1. Grundlagen: Wie Computer „denken“

1.1 Alles ist binär

Ein Computer versteht nur **0 und 1** (Bits). 8 Bits sind ein **Byte**. Alles, was er speichert, ist als Zahlen codiert:

- **Text:** Jedes Zeichen bekommt über eine Codetabelle eine Nummer. In **ASCII** ist `'A'` = 65 = `01000001` (binär).
- **Bilder:** bestehen aus **Pixeln**. Jedes Pixel hat im **RGB-Modell** drei Farbkanäle mit je 8 Bit (0 bis 255). Reines Rot: R=255 (`11111111`), G=0 (`00000000`), B=0 (`00000000`).

1.2 Programmiersprachen

Maschinencode (Nullen und Einsen) ist für Menschen unlesbar. Deshalb gibt es Programmiersprachen:

Ebene	Beispiel	Eigenschaft
Maschinencode	<code>01010011</code> <code>01111001 ...</code>	direkt von der Hardware ausführbar, keine Abstraktion
Low-Level	Assembly	lesbar, aber jeder Befehl entspricht fast genau einem Maschinenbefehl
High-Level	Java, C++, Python	hohe Abstraktion: <code>System.out.println("Hi")</code> erledigt alles

Abstraktion heißt: Details verstecken, damit man sich aufs Wesentliche konzentrieren kann.

1.3 Kompilieren vs. Interpretieren

	Kompiliert (z. B. C++)	Interpretiert (z. B. Python)
Ausführung	vorher komplett in Maschinencode übersetzt	Zeile für Zeile zur Laufzeit
Geschwindigkeit	meist schneller	meist langsamer
Portabilität	plattformspezifisch	portabler
Fehlererkennung	beim Kompilieren	erst zur Laufzeit

➡ **Java macht beides:** Der Code wird zuerst zu **Bytecode** kompiliert, dieser wird dann von der **JVM** interpretiert bzw. JIT-kompiliert (Details in Kapitel 3).

1.4 Syntax vs. Semantik

- **Syntax** = die Grammatikregeln. Ist der Code *formal korrekt geschrieben*? Fehlt ein `;` oder ist ein Wort falsch geschrieben, meldet der **Compiler** einen Fehler.
- **Semantik** = die Bedeutung. *Tut* der Code das, was ich will? Syntaktisch korrekter Code kann trotzdem das Falsche tun. Das nennt man einen **Bug**.

Anders als natürliche Sprache ist eine Programmiersprache **nie mehrdeutig**: Ein gültiger Ausdruck hat genau eine Bedeutung.

```
int durchschnitt = (3 + 4) / 2; // Syntax ✓, Semantik ✗: Ergebnis ist 3, n
```

✓ Selbsttest 1

1. Was ist der Unterschied zwischen Syntax- und Semantikfehler? Wer findet welchen?
2. Warum ist Java „kompiliert *und* interpretiert“?

▼ Lösung

1. Ein Syntaxfehler verletzt die Grammatik der Sprache, der **Compiler** findet ihn. Ein Semantikfehler ist ein Logikfehler (Bug): Das Programm läuft, tut aber das Falsche. Den musst **du** finden, durch Testen und Debuggen.
2. `javac` kompiliert den Quellcode zu Bytecode (`.class`). Die JVM führt diesen Bytecode aus, indem sie ihn interpretiert und häufig genutzte Teile per JIT-Compiler in Maschinencode übersetzt.

2. Computational Thinking

Bevor du programmierst, musst du das Problem verstehen und einen Lösungsweg haben.

Computational Thinking besteht aus vier Techniken:

Technik	Bedeutung
Decomposition (Zerlegung)	großes Problem in kleine, handliche Teilprobleme zerlegen
Pattern Recognition (Mustererkennung)	Gemeinsamkeiten und Wiederholungen erkennen
Abstraction (Abstraktion)	nur das Wichtige betrachten und die Lösung verallgemeinern
Algorithms (Algorithmus)	eine Schritt-für-Schritt-Anleitung zur Lösung formulieren

Beispiel aus der Vorlesung: Summe $1 + 2 + \dots + 200$ im Kopf

1. **Zerlegung:** Nicht der Reihe nach addieren, sondern Paare von außen bilden: $1+200$, $2+199$, $3+198$...
2. **Muster:** Jedes Paar ergibt **201**. Es gibt $200 / 2 = 100$ Paare.
3. **Algorithmus:** $100 \cdot 201 = 20\ 100$.
4. **Abstraktion:** Für jede Zahl x gilt: $\text{Summe} = (x / 2) \cdot (x + 1)$.

Pseudocode ist eine Beschreibung ohne feste Syntax (Befehle GROSS, Variablen klein):

```
OUTPUT 'What is the number to be added?'
INPUT user inputs the number
STORE the user's input in the number variable
STORE (number / 2) * (number + 1) in total_sum
OUTPUT 'The sum of numbers from 1 to ' + number + ' is ' + total_sum
```

Oder ohne den Gauß-Trick, mit einer Schleife:

```
STORE 0 in total_sum
INPUT end
FOR number FROM 1 TO end DO
    STORE total_sum + number in total_sum
OUTPUT total_sum
```

Und so sieht die Schleifenvariante in Java aus (vorgreifend auf Kapitel 8):

```
int end = 200;
int totalSum = 0;
for (int number = 1; number <= end; number++) {
    totalSum = totalSum + number;
}
System.out.println("The sum of numbers from 1 to " + end + " is " + totalSum);
```

⚠ **Java-Falle:** Die Formel $(x / 2) * (x + 1)$ liefert in Java für **ungerade** x mit `int` ein falsches Ergebnis, weil $x / 2$ abgerundet wird (siehe Kapitel 6). Besser: $x * (x + 1) / 2$.

Merksatz: Computational Thinking entscheidet, *welche* Schritte nötig sind.
Programmieren sagt dem Computer, *wie* er sie ausführt.

✳ Aufgabe: Handshake Summit of Sillyville

50 Personen geben einander **genau einmal** die Hand. Wie viele Handschläge gibt es? Löse die Aufgabe mit Zerlegung, Mustererkennung und Abstraktion.

▼ Lösungsweg

- **Zerlegen:** Person 1 schüttelt 49 Hände. Person 2 hat Person 1 schon begrüßt, also nur noch 48 neue. Person 3 noch 47 neue ...
- **Muster:** $49 + 48 + 47 + \dots + 1$
- **Abstraktion:** Summe von 1 bis $(n-1) = n \cdot (n - 1) / 2$
- **Ergebnis:** $50 \cdot 49 / 2 = 1225$

Pseudocode:

```
INPUT n
STORE n * (n - 1) / 2 in handshakes
OUTPUT handshakes
```

Java-Lösung (Schleife und Formel): `beispiele/Handshakes.java`

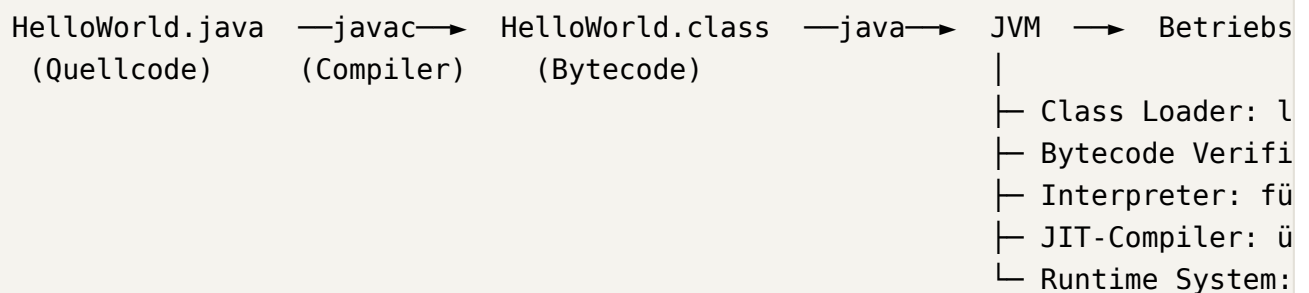
3. Java, JVM & dein erstes Programm

3.1 Was ist Java?

Java ist zwei Dinge: 1. eine **Programmiersprache**: allgemein einsetzbar, klassenbasiert, objektorientiert, Syntax ähnlich wie C 2. eine **Plattform**: die **Java-API** (fertige Bibliotheken, in *Packages* organisiert) plus die **JVM** (Java Virtual Machine)

Varianten: **Java SE** (Standard Edition, für Desktop und Server; das nutzen wir), Jakarta EE (Enterprise), Java ME/Card (Kleingeräte), JavaFX (grafische Oberflächen, kommt später im Kurs).

3.2 Vom Quellcode zur Ausführung



Vorteil: Der gleiche Bytecode läuft auf jedem System mit JVM (Windows, Mac, Linux): „Write once, run anywhere“.

3.3 Dein erstes Programm

Datei `HelloWorld.java`:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Im Terminal:

```
java --version          # prüft, ob Java installiert ist (sollte 21 zeigen)
javac HelloWorld.java   # kompiliert -> erzeugt HelloWorld.class
java HelloWorld         # startet das Programm (ohne .class!)
```

Ausgabe: `Hello, World!`

In **VS Code** (mit dem „Extension Pack for Java“) reicht ein Klick auf **Run** über der `main`-Methode.

3.4 Aufbau eines Java-Programms, Zeile für Zeile

```
package com.example.sample;           // (1) Package: wo liegt die Datei?

public class HelloWorld {              // (2) Klasse: Name == Dateiname!
    // Main method - Entry point       // (3) Kommentar
    public static void main(String[] args) { // (4) main-Methode: hier starte
        System.out.println("Hello, World!"); // (5) Anweisung, endet mit ;
    }                                   // (6) Block-Ende
}
```

Nr.	Regel
(1)	Das Package muss zur Ordnerstruktur passen: <code>com.example.sample</code> bedeutet Ordner <code>com/example/sample/</code> . Für einfache Übungen kannst du das Package weglassen.
(2)	Der Klassenname muss exakt dem Dateinamen entsprechen: <code>HelloWorld</code> steht in <code>HelloWorld.java</code> .
(4)	Ein eigenständig startbares Programm braucht <code>public static void main(String[] args)</code> . Die JVM lädt die Klasse, sucht <code>main</code> und führt sie aus. Fehlt <code>main</code> , gibt es einen Fehler.
(5)	Die meisten Anweisungen enden mit <code>;</code> . Pro Zeile nur eine Anweisung.
(6)	Blöcke <code>{ ... }</code> fassen zusammengehörige Anweisungen zusammen. Jede <code>{</code> braucht eine passende <code>}</code> .

⚠ **Java ist case-sensitive:** `HelloWorld` $\neq$ `helloworld`, `System` $\neq$ `system`, `String` $\neq$ `string`.

3.5 Kommentare

```
// einzeliger Kommentar

/*
 * mehrzeiliger Kommentar
 */

/**
 * JavaDoc-Kommentar: beschreibt Klassen und Methoden.
 * @param args Kommandozeilenargumente
 */
```

Java ignoriert Kommentare, Leerzeilen und Einrückung. Sie sind nur für Menschen da, aber für Menschen **sehr** wichtig.

3.6 Coding-Konventionen (bitte einhalten, das wird bewertet!)

Was	Konvention	Beispiel
Klassen	groß beginnen, UpperCamelCase	HelloWorld, AreaCircum
Methoden und Variablen	klein beginnen, lowerCamelCase	main, circleArea, printLine
Konstanten	nur GROSSBUCHSTABEN (mit <code>_</code>)	PI, MAX_POINTS
Packages	nur kleinbuchstaben	com.example.sample
Namen	aussagekräftig, mindestens 3 Zeichen, englisch	counter statt c oder zaehler
Einrückung	jeder Block wird eingerückt (Tab = 4 Leerzeichen)	
Klammern	einheitlicher Stil: <code>{</code> in derselben Zeile <i>oder</i> in der nächsten	

Ausnahme: Kurze Zählvariablen in Schleifen wie `i`, `j` sind allgemein üblich.

3.7 IDE (Integrated Development Environment)

Eine IDE bündelt **Editor** (Syntax-Highlighting, Autovervollständigung, Fehler beim Tippen), **Compiler**, **Debugger** (Programm anhalten, Variablen ansehen), **Projektverwaltung** und **Terminal**. Im Kurs verwenden wir **VS Code**. Alternativen sind IntelliJ IDEA, Eclipse und NetBeans.

✓ Selbsttest 3

1. Deine Datei heißt `Rechner.java`, darin steht `public class Calculator`. Was passiert?
2. Welche Datei erzeugt `javac` und wer führt sie aus?
3. Was ist falsch? `public static void Main(String[] args)`

▼ Lösung

1. Compilerfehler: Eine `public` Klasse muss in einer gleichnamigen Datei stehen (`Calculator.java`).
2. Eine `.class`-Datei mit Bytecode. Die **JVM** führt sie aus (Befehl `java`).
3. `Main` ist großgeschrieben. Die JVM sucht `main` (klein), also findet sie keinen Einstiegspunkt.

4. Datentypen, Variablen, Konstanten & Casting

4.1 Typen und Literale

- In Java hat **alles einen Typ**. Der Typ bestimmt, wie viel Speicher gebraucht wird und welche Operationen erlaubt sind („5 hoch Kürbis“ geht nicht).
- Ein **Literal** ist ein fester Wert im Code: `42`, `3.14`, `true`, `'A'`, `"Hallo"`.
- Manche Operatoren verhalten sich je nach Typ anders: `+` addiert Zahlen, **verkettet** aber Strings.

4.2 Übersicht der Datentypen

```
Datentyp
├─ Primitive Typen (speichern direkt den Wert)
│   ├── boolean                true / false
│   └─ numerisch
│       ├── ganzzahlig: byte, short, int, long, (char)
│       └─ Gleitkoma: float, double
└─ Referenztypen (verweisen auf ein Objekt)
    ├── Klassen                z. B. String, Scanner
    ├── Arrays                  (Block 5)
    └─ Interfaces              (Block 9)
```

4.3 Primitive Typen im Detail

Typ	Größe	Wertebereich	Literal-Beispiel	Wann verwenden?
<code>byte</code>	8 Bit	-128 ... 127	<code>89</code>	selten, bei Speicherknappheit
<code>short</code>	16 Bit	-32 768 ... 32 767	<code>1000</code>	selten
<code>int</code>	32 Bit	ca. -2,1 Mrd ... +2,1 Mrd	<code>42</code> , <code>-1</code>	Standard für ganze Zahlen
<code>long</code>	64 Bit	ca. $\pm 9 \cdot 10^{18}$	<code>3456789L</code>	sehr große Zahlen (z. B. Fakultät)
<code>float</code>	32 Bit	$\pm 3,4 \cdot 10^{38}$, ~7 Stellen genau	<code>32.5f</code>	selten
<code>double</code>	64 Bit	$\pm 1,7 \cdot 10^{308}$, ~15 Stellen genau	<code>3.14</code>	Standard für Kommazahlen
<code>boolean</code>	-	<code>true</code> , <code>false</code>	<code>true</code>	Bedingungen, Ja/Nein
<code>char</code>	16 Bit	ein Unicode-Zeichen	<code>'f'</code> , <code>'?'</code>	einzelnes Zeichen

⚠ **Suffixe beachten:** `long big = 3456789L;` und `float f = 32.5f;`. Ohne `f` ist `32.5` ein `double` und passt nicht in `float`.

⚠ **Anführungszeichen:** `char` verwendet einfache `'a'`, `String` doppelte `"abc"`.

ℹ Ein `char` belegt in Java 16 Bit (UTF-16-Codeeinheit). Die Folie spricht von „UTF-8“; gemeint ist, dass Java Unicode verwendet.

Wichtige Nicht-Primitive: - `String`: eine Zeichenkette aus 0 bis beliebig vielen Zeichen, z. B. `"Hello"`. Achtung, großes **S**! - `enum`: eine feste Menge eigener Werte, z. B. `RED`, `GREEN`, `BLUE`.

4.4 Variablen

Eine **Variable** ist ein **Name für einen Speicherbereich**, in dem ein Wert eines bestimmten Typs liegt.

Java ist **statisch typisiert**: Der Typ jeder Variable steht beim Kompilieren fest und kann sich **nie ändern**.

```
int x = 1;
x = "Hey! Now I am a string!"; // ❌ Compilerfehler! (In Python ginge das, d
```

Deklaration (Variable anlegen) und **Initialisierung** (ersten Wert zuweisen):

```
datentyp name;           // Deklaration
datentyp name = wert;    // Deklaration + Initialisierung

int age;                 // deklariert, hat noch keinen Wert
age = 21;                // Zuweisung
double area = 3.5;
boolean isFun = true;
char letterGrade = 'A';
String name = "Anna";
int a = 2, b = 3;        // mehrere auf einmal (erlaubt, aber sparsam verwe
int z = a;               // Wert von a wird in z KOPIERT
```

⚠ Eine lokale Variable **muss vor dem ersten Lesen einen Wert haben**, sonst meldet der Compiler `variable might not have been initialized`.

4.5 Bezeichner (Identifizier): Regeln für Namen

Namen für Variablen, Konstanten, Methoden, Klassen, Packages: - bestehen aus Buchstaben, Ziffern und `_` (auch `$` ist erlaubt, aber unüblich) - dürfen **nicht mit einer Ziffer beginnen**: `2teZahl` ❌, `zahl2` ✅ - sind **case-sensitive**: `letterGrade` und `lettergrade` sind zwei verschiedene Variablen! - dürfen **keine reservierten Schlüsselwörter** sein, z. B.:

```
abstract boolean break byte case catch char class const continue default do double
else enum extends final finally float for goto if implements import instanceof int
interface long native new package private protected public return short static
strictfp super switch synchronized this throw throws transient try void volatile
while
```

4.6 Konstanten

Eine Konstante ist eine Variable, deren Wert sich **nicht ändert**. Sie macht Code lesbarer, weil feste Werte einen Namen bekommen.

```
final datentyp NAME = wert;
```

```
final int SPEED_OF_LIGHT = 300;
```

```
final double PI = 3.142;
```

```
final String COMPANY = "acme";
```

```
PI = 3; // ❌ Compilerfehler: cannot assign a value to final variable PI
```

💡 Für π gibt es schon eine fertige Konstante: `Math.PI`.

4.7 Type Casting (Typumwandlung)

Implizit (automatisch): von klein nach groß. Das ist sicher, weil keine Information verloren geht.

```
byte x = 24;  
int y = x;           // ✅ automatisch  
double d = y;        // ✅ int -> double, d = 24.0
```

Reihenfolge: `byte → short → int → long → float → double`

Explizit (du musst casten): von groß nach klein. Dabei kann Information verloren gehen.

```
int x = 24;  
byte b = x;           // ❌ Fehler: "incompatible types: possible lossy conversion  
byte b = (byte) x;    // ✅ "Ich weiß, was ich tue"
```

Frage von der Folie: 24 passt doch in ein byte, warum geht `byte b = x;` nicht?

➡ Der Compiler prüft den **Typ**, nicht den aktuellen Wert. `x` ist ein `int` und *könnte* zur Laufzeit auch 1 000 000 enthalten. Deshalb verlangt Java die explizite Bestätigung `(byte)`.

```
double price = 9.99;  
int euros = (int) price;           // 9, abgeschnitten, NICHT gerundet!  
long rounded = Math.round(price); // 10, so wird gerundet
```

String → Zahl (kein Cast, sondern eine Methode):

```
String text = "123";  
int number = Integer.parseInt(text);           // 123  
double value = Double.parseDouble("3.5");      // 3.5  
  
int bad = Integer.parseInt("Hello World");    // 🌟 Laufzeitfehler: NumberForm
```

`Integer.valueOf(text)` funktioniert für uns genauso wie `Integer.parseInt(text)`.

✅ Selbsttest 4

1. Welcher Typ für: Alter, Kontostand, „ist volljährig“, Anfangsbuchstabe, Nachname?
2. Kompiliert das? `float f = 1.5;`
3. Was ergibt `(int) 7.9`?
4. Ist `my-var` ein gültiger Variablenname? Und `_count2`? Und `class`?

▼ Lösung

1. `int age`, `double balance`, `boolean isAdult`, `char initial`, `String lastName`
2. Nein: `1.5` ist ein `double`. Richtig ist `float f = 1.5f;`
3. `7`: Beim Cast wird abgeschnitten, nicht gerundet.
4. `my-var` ❌ (Bindestrich wird als Minus gelesen), `_count2` ✅, `class` ❌
(Schlüsselwort)

5. Ein- und Ausgabe auf der Konsole

5.1 Ausgabe

```
System.out.println(ausgabe); // gibt aus und macht danach einen Zeilenumbruch
System.out.print(ausgabe);   // gibt aus, der Cursor bleibt in der Zeile
System.out.println();        // gibt nur eine leere Zeile aus
```

Was man ausgeben kann:

```
int taxRate = 20;
int radius = 2;

System.out.println("Hello World!"); // Literal (String)
System.out.println(12.5);           // Literal (Zahl)
System.out.println(2 * Math.PI * radius); // Ausdruck: das Ergebnis wird
System.out.println(taxRate);         // Variable: ihr Wert wird ausgeben
System.out.println("The tax rate is " + taxRate + '%'); // Verkettung mit +
```

```
System.out.print("The tax rate is ");
System.out.print(taxRate);
System.out.println();
// Ausgabe in EINER Zeile: The tax rate is 20
```

  **mit Strings wird von links nach rechts ausgewertet:**

```
System.out.println("Sum: " + 1 + 2); // Sum: 12 ("Sum: 1" + 2)
System.out.println("Sum: " + (1 + 2)); // Sum: 3 (Klammer zuerst)
System.out.println(1 + 2 + " is it"); // 3 is it (erst 1+2, dann Text)
```

 Nützliches Extra für schöne Zahlen: `System.out.printf("%.2f%n", 3.14159);` gibt `3.14` aus.

5.2 Eingabe mit Scanner

```
import java.util.Scanner; // (1) ganz oben in der Datei, vor der

public class InputDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // (2) EINEN Scanner anlegen

        System.out.print("Name: ");
        String name = scanner.nextLine(); // (3) ganze Zeile als String

        System.out.print("Alter: ");
        int age = scanner.nextInt(); // ganze Zahl

        System.out.print("Gehalt: ");
        double salary = scanner.nextDouble(); // Kommazahl

        System.out.println(name + " ist " + age + " und verdient " + salary);
    }
}
```

Methode	liest	Rückgabotyp
<code>next()</code>	ein Wort (bis zum nächsten Leerzeichen)	<code>String</code>
<code>nextLine()</code>	die ganze Zeile (bis Enter)	<code>String</code>
<code>nextInt()</code>	eine ganze Zahl	<code>int</code>
<code>nextDouble()</code>	eine Kommazahl	<code>double</code>

⚠ Die zwei häufigsten Scanner-Fallen:

1. **Falsche Eingabe führt zum Absturz:** Tippt jemand bei `nextInt()` den Text „abc“, gibt es eine `InputMismatchException`.
2. `nextInt()` **gefolgt von** `nextLine()`: `nextInt()` liest nur die Zahl, das Enter bleibt im Puffer. Das nächste `nextLine()` liefert dann sofort einen **leeren String**.

`java int age = scanner.nextInt(); scanner.nextLine();` // ✓ Lösung: übrig gebliebenes Enter "wegessen" `String city = scanner.nextLine();` Alternative: immer `nextLine()` lesen und umwandeln: `int age = Integer.parseInt(scanner.nextLine());`

⚠ Bei `nextDouble()` hängt das Dezimaltrennzeichen von der Systemsprache ab: Auf deutschem System erwartet Java `3,5` statt `3.5`.

5.3 Komplettbeispiel: Kreisrechner (Vorgehen wie in der Vorlesung)

Schritt 1: Algorithmus überlegen 1. Begrüßung ausgeben 2. Radius abfragen und einlesen 3. Fläche = $\pi \cdot r \cdot r$ 4. Umfang = $2 \cdot \pi \cdot r$ 5. Ergebnisse ausgeben

Schritt 2: benötigte Daten festlegen `radius`: `int` (Eingabe) · `area`, `circumference`: `double` (Ausgabe) · `PI`: `double`-Konstante

Schritt 3: Gerüst mit Kommentaren schreiben, dann ausfüllen

```
import java.util.Scanner;

/**
 * AreaCircum - computes the area & circumference of a circle given its radius
 */
public class AreaCircum {
    public static void main(String[] args) {
        // Konstanten
        final double PI = 3.142;

        // Variablen
        int radius;
        double area;
        double circumference;
        Scanner scan = new Scanner(System.in);

        // 1. Begrüßung
        System.out.println("Welcome to the Area Circumference Calculator");
        // 2. Radius einlesen
        System.out.println("Please enter the radius: ");
        radius = scan.nextInt();
        // 3. Fläche
        area = PI * radius * radius;
        // 4. Umfang
        circumference = 2 * PI * radius;
        // 5. Ausgabe
        System.out.println("The area of a circle of radius: " + radius + " is");
        System.out.println("and its circumference is: " + circumference);
    }
}
```

 **Diese Arbeitsweise solltest du dir angewöhnen:** Zuerst die Schritte als Kommentare hinschreiben, dann Schritt für Schritt Code darunter ergänzen. So verlierst du

nie den Überblick.

6. Operatoren & Ausdrücke

Ein **Ausdruck** (Expression) ist alles, was zu einem **Wert** ausgewertet werden kann: `5`, `x`, `a + b`, `age >= 18`, `Math.sqrt(16)`.

Eine **Zuweisung** hat die Form `ergebnisVariable = ausdruck;`. Der Typ des Ausdrucks muss zur Variable passen.

6.1 Arithmetische Operatoren

Operator	Bedeutung	Beispiel	Ergebnis
<code>+</code>	Addition (bei Strings: Verkettung)	<code>7 + 2</code>	<code>9</code>
<code>-</code>	Subtraktion	<code>7 - 2</code>	<code>5</code>
<code>*</code>	Multiplikation	<code>7 * 2</code>	<code>14</code>
<code>/</code>	Division	<code>7 / 2</code>	<code>3 (!)</code>
<code>%</code>	Modulo (Rest der Division)	<code>7 % 2</code>	<code>1</code>

6.2 ⚠ DIE Falle: Ganzzahldivision

`int / int` ergibt immer `int`: Die Nachkommastellen werden abgeschnitten.

```
System.out.println(7 / 2);           // 3
System.out.println(7 / 2.0);         // 3.5 (ein double beteiligt, also double-Erg
System.out.println(1 / 3);           // 0
double avg = (3 + 4) / 2;             // 3.0 (!) Erst wird int/int = 3 berechnet, DA
double avg2 = (3 + 4) / 2.0;          // 3.5 ✓
```

Aufgabe von der Folie: `int z = 4 + 2 / 3 - 1;` Welchen Wert hat z?

→ `2 / 3 = 0` (Punkt vor Strich, Ganzzahldivision), dann `4 + 0 - 1 = 3`

Achtung, auch das Folienbeispiel hat diesen Bug:

```
final int TAXRATE = 20;
int gross = 234;
double net = gross * (1 - TAXRATE / 100); // 20/100 = 0 -> net = 234.0 ✗ (
double net = gross * (1 - TAXRATE / 100.0); // 0.2 -> net = 187.2 ✓
```

i Kommazahlen sind im Computer nicht immer exakt. `234 * 0.8` ergibt `187.20000000000002`. Das ist normal. Für schöne Ausgabe: `System.out.printf("%.2f%n", net);`

6.3 Rechenregeln (Priorität)

1. **Klammern** zuerst
2. `*`, `/`, `%` (Punkt vor Strich)
3. `+`, `-`
4. bei gleicher Stufe: **von links nach rechts**

Im Zweifel: **Klammern setzen!** Das macht den Code auch lesbarer.

6.4 Modulo `%`: der unterschätzte Operator

```
7 % 2    // 1
5 % 3    // 2
8 % 4    // 0
1 % 2    // 1
```

Typische Einsätze:

Frage	Code
Ist n gerade?	<code>n % 2 == 0</code>
Ist n ungerade?	<code>n % 2 != 0</code>
Ist n durch 400 teilbar?	<code>n % 400 == 0</code>
Letzte Ziffer von n	<code>n % 10</code>
n ohne letzte Ziffer	<code>n / 10</code>
Schaltjahr?	<code>(year % 4 == 0 && year % 100 != 0) year % 400 == 0</code>

6.5 Zuweisungs- und Inkrement-Operatoren

Kurzform	bedeutet
<code>x += 5;</code>	<code>x = x + 5;</code>
<code>x -= 5;</code>	<code>x = x - 5;</code>
<code>x *= 2;</code>	<code>x = x * 2;</code>
<code>x /= 2;</code>	<code>x = x / 2;</code>
<code>x %= 3;</code>	<code>x = x % 3;</code>
<code>x++;</code>	<code>x = x + 1;</code>
<code>x--;</code>	<code>x = x - 1;</code>

`i++` (Post-Inkrement) liefert den **alten** Wert und erhöht danach. `++i` (Prä-Inkrement) erhöht **zuerst**. Als alleinstehende Anweisung (`i++;`) ist beides gleich. Unterschiede gibt es nur, wenn man den Wert gleichzeitig verwendet:

```
int i = 5;
int post = i++; // post = 5, i = 6
int pre = ++i;  // i = 7, pre = 7
```

6.6 Vergleichsoperatoren (Ergebnis ist immer `boolean`)

<code>==</code>	<code>!=</code>	<code><</code>	<code><=</code>	<code>></code>	<code>>=</code>
gleich	ungleich	kleiner	kleiner gleich	größer	größer gleich

⚠ `=` ist eine **Zuweisung**, `==` ist ein **Vergleich**. `if (x = 5)` ist in Java ein Compilerfehler.

6.7 Logische Operatoren

Operator	Name	true, wenn ...
<code>&&</code>	UND	beide Seiten true sind
<code> </code>	ODER	mindestens eine Seite true ist
<code>!</code>	NICHT	der Ausdruck false ist (dreht um)

Wahrheitstabelle:

a	b	a && b	a b	!a
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true
false	false	false	false	true

```
boolean isEligible = age >= 18 && hasID;  
boolean outOfRange = x < 5 || x > 10;  
boolean isDigit = aChar >= '0' && aChar <= '9';  
boolean notFound = !found;
```

Short-Circuit (Kurzschluss): Java wertet von links aus und hört auf, sobald das Ergebnis feststeht. Bei `false && ...` wird die rechte Seite nie ausgewertet, bei `true || ...` ebenfalls nicht. Das nutzt man bewusst:

```
boolean isValid = input != null && input.length() > 0; //  sicher
```

 Auf der Folie steht `input != null || input.length() > 0`. Das ist **fehlerhaft**: Ist `input` gleich `null`, ist die linke Seite `false`, also wird die rechte ausgewertet, und `null.length()` stürzt ab (`NullPointerException`). Richtig ist `&&`.

 Mathematische Kettenvergleiche gibt es in Java nicht: `0 <= x <= 10` , sondern `x >= 0 && x <= 10` .

6.8 Ternärer Operator (Kurz-if für Werte)

```
bedingung ? wertWennTrue : wertWennFalse
```

```
String result = (grade >= 50) ? "Pass" : "Fail";  
int max = (a > b) ? a : b;
```

6.9 Nützliche Math-Methoden

Methode	Bedeutung	Beispiel
<code>Math.sqrt(x)</code>	Wurzel (liefert <code>double</code>)	<code>Math.sqrt(16)</code> ergibt <code>4.0</code>
<code>Math.pow(x, y)</code>	x hoch y (liefert <code>double</code>)	<code>Math.pow(2, 3)</code> ergibt <code>8.0</code>
<code>Math.abs(x)</code>	Betrag	<code>Math.abs(-5)</code> ergibt <code>5</code>
<code>Math.max(a, b)</code> / <code>Math.min(a, b)</code>	Maximum / Minimum	<code>Math.max(3, 7)</code> ergibt <code>7</code>
<code>Math.round(x)</code>	runden	<code>Math.round(2.5)</code> ergibt <code>3</code>
<code>Math.PI</code>	π (Konstante, keine Methode)	<code>3.14159...</code>

✓ Selbsttest 6

Welchen Wert und Typ haben diese Ausdrücke? 1. `10 / 4` 2. `10 / 4.0` 3. `10 % 4` 4. `2 + 3 * 4` 5. `(2 + 3) * 4` 6. `"A" + 1 + 2` 7. `5 > 3 && 2 > 4` 8. `!(5 == 5)`

▼ Lösung

1. `2` (int) 2. `2.5` (double) 3. `2` (int) 4. `14` (int) 5. `20` (int) 6. `"A12"` (String) 7. `false` (boolean) 8. `false` (boolean)

7. Verzweigungen: if, else, switch

Bisher liefen Programme stur von oben nach unten. Mit **Kontrollstrukturen** steuern wir den Ablauf: - **Verzweigungen** (Conditionals): *zwischen* Wegen wählen, mit `if / else` und `switch` - **Schleifen** (Iterations): Abschnitte *wiederholen* (Kapitel 8)

7.1 `if`

```
if (bedingung) {  
    // wird nur ausgeführt, wenn bedingung true ist  
}
```

Die **Bedingung** muss ein `boolean`-Ausdruck sein: - eine boolean-Variable: `isOver21`, `found` - eine Methode, die boolean liefert: `input.equals("yes")`, `isEven(n)` - ein Vergleich: `age >= 21`, `speed == 0`, `year % 4 != 0` - eine logische Verknüpfung: `height > 2 && weight <= 80`

```
if (x > 0) {  
    System.out.println("the value of x is positive");  
}
```

7.2 `if ... else`

```
if (bedingung) {  
    // wenn true  
} else {  
    // wenn false (optional)  
}
```

```
if (y == 0) {  
    System.out.println("Error: can't divide by zero");  
} else {  
    z = x / y;  
    System.out.println("The result is " + z);  
}
```

⚠ **Immer geschweifte Klammern verwenden!** Ohne `{}` gehört nur **eine** Anweisung zum `if`:

```
if (x > 0)
    System.out.println("positiv");
    System.out.println("das wird IMMER ausgegeben!");    // gehört NICHT zum if
```

⚠️ Kein Semikolon nach der Bedingung: `if (x > 0);` beendet das `if` sofort, der Block danach läuft immer.

7.3 `else if` : mehrere Alternativen

```
if (x < 0) {
    System.out.println("negative");
} else if (x == 0) {                // hier gilt schon: x >= 0
    System.out.println("zero");
} else {                           // hier gilt: x > 0
    System.out.println("positive");
}
```

Es wird **genau ein** Zweig ausgeführt: der **erste**, dessen Bedingung true ist. Danach springt Java ans Ende der ganzen Kette.

7.4 Wichtig: Kette vs. einzelne ifs

<i>// Variante A: else-if-Kette</i>	<i>// Variante B: einzelne ifs</i>
<code>if (c1) print("A");</code>	<code>if (c1) print("A");</code>
<code>else if (c2) print("B");</code>	<code>if (c2) print("B");</code>
<code>else if (c3) print("C");</code>	<code>if (c3) print("C");</code>
<code>else print("D");</code>	<code>if (c4) print("D");</code>

- **A:** Höchstens ein Buchstabe wird ausgegeben. Genauer: **genau einer**, denn das `else` fängt alles Übrige auf.
- **B:** Jede Bedingung wird unabhängig geprüft. Es können **0 bis 4** Buchstaben ausgegeben werden.

Praxisbeispiel Notenvergabe: Die **Reihenfolge** zählt! Von der strengsten Bedingung zur schwächsten prüfen:

```

if (percentage >= 90) {
    System.out.println("Excellent");
} else if (percentage >= 75) {           // hier ist percentage garantiert < 90
    System.out.println("Good job");
} else if (percentage >= 60) {
    System.out.println("Passed");
} else {
    System.out.println("Needs improvement");
}

```

Mit einzelnen `if`s würde bei 95 % „Excellent“, „Good job“ **und** „Passed“ ausgegeben.

7.5 Verschachtelte ifs

```

if (x >= 0) {
    if (x == 0) {
        System.out.println("zero");
    } else {
        System.out.println("positive");
    }
} else {
    System.out.println("negative");
}

```

Das funktioniert, aber `else if` ist meist übersichtlicher.

7.6 Klassische Muster

Betrag der Differenz, drei gleichwertige Wege (oder einfach `Math.abs(x - y)`):

```

if (x > y) {
    z = x - y;
} else {
    z = y - x;
}

```

Minimum von drei Zahlen:

```

int min;
if (first < second) {
    min = first;
} else {
    min = second;
}
if (third < min) {
    min = third;
}

```

💡 Verallgemeinerung: „Kandidat merken, jeden weiteren Wert vergleichen und ggf. ersetzen“. Genau dieses Muster verwendest du später mit Schleifen und Arrays für beliebig viele Werte.

7.7 Strings vergleichen: immer mit `equals` !

```

String input = scanner.nextLine();

if (input.equals("a string")) { ... }           // ✓ vergleicht den INHALT
if (input.equalsIgnoreCase("YES")) { ... }      // ✓ ignoriert Groß-/Kleinschr
if (input == "a string") { ... }                // ✗ vergleicht, ob es DASSELBE

```

Warum? `String` ist ein **Referenztyp**. `==` vergleicht bei Referenztypen, ob beide Variablen auf *dasselbe Objekt* zeigen, nicht ob der Text gleich ist. Mit eingelesenen Strings ist das praktisch immer `false`. Bei primitiven Typen (`int`, `char`, ...) ist `==` richtig.

7.8 `switch`

Praktisch, wenn **eine** Variable mit **festen Einzelwerten** verglichen wird.

```

switch (ausdruck) {
    case wert1:
        anweisungen;
        break;           // verlässt den switch
    case wert2:
        anweisungen;
        break;
    default:              // optional: wenn kein case passt
        anweisungen;
}

```

- Der Ausdruck darf vom Typ `int`, `char`, `short`, `byte`, `String` oder ein `enum` sein. **Nicht** erlaubt: `double`, `boolean`, `long`.
- `case` braucht **konstante Einzelwerte** (`1`, `'+'`, `"Run"`), **keine** Bedingungen wie `case x > 5` ❌.
- ⚠️ **Fall-through:** Fehlt `break`, werden **alle folgenden** cases (inkl. `default`) mit ausgeführt!

```
char operator = '+';
int x = 6, y = 3, z = 0;

switch (operator) {
    case '+':
        z = x + y;
        break;
    case '-':
        z = x - y;
        break;
    case '*':
        z = x * y;
        break;
    case '/':
        z = x / y;
        break;
    default:
        System.out.println("invalid operator passed!");
}
```

Fall-through bewusst genutzt (mehrere Werte, gleiche Aktion):

```
switch (month) {
    case 12:
    case 1:
    case 2:
        System.out.println("Winter");
        break;
    // ...
}
```

💡 **Moderne Schreibweise (seit Java 14, in Java 21 verfügbar):** mit `->` gibt es kein Fall-through und kein `break`:

```

switch (operator) {
    case '+' -> z = x + y;
    case '-' -> z = x - y;
    case '*', 'x' -> z = x * y;    // mehrere Werte
    default -> System.out.println("invalid operator passed!");
}

```

Im Kurs wird die klassische Form mit `break` gezeigt. Beide solltest du lesen können.

✓ Selbsttest 7

1. Was gibt dieser Code bei `x = 5` aus? `java if (x > 3) System.out.println("A"); if (x > 4) System.out.println("B"); else System.out.println("C");`
2. Was gibt dieser `switch` bei `n = 2` aus? `java switch (n) { case 1: System.out.println("eins"); case 2: System.out.println("zwei"); case 3: System.out.println("drei"); default: System.out.println("?"); }`
3. Warum ist `if (name == "Anna")` gefährlich?

▼ Lösung

1. `A` und `B`. Die beiden ifs sind unabhängig. Das `else` gehört nur zum zweiten if.
2. `zwei`, `drei`, `?`: kein `break`, also Fall-through bis zum Ende.
3. `==` vergleicht Referenzen, nicht den Inhalt. Richtig: `name.equals("Anna")`.

8. Schleifen

Schleifen **wiederholen** Anweisungen, ohne dass du den Code mehrfach schreiben musst: - **Automatisierung** (z. B. Zahlen von 1 bis 100 ausgeben) - Berechnungen mit wiederholten Schritten (Summe, Fakultät) - kürzerer, lesbarer Code ohne Duplikate - wenn die Anzahl der Wiederholungen **vorher nicht bekannt** ist (z. B. „bis der User 0 eingibt“)

8.1 Die drei Zutaten jeder Schleife

```
int i = 0; // 1. INITIALISIERUNG: Startwert
while (i < 5) { // 2. BEDINGUNG: Wie lange läuft die Schl
    System.out.println("Hello World!");
    i = i + 1; // 3. UPDATE: Variable verändern
}
```

Vergessen ...	Folge
Initialisierung	Compilerfehler oder unvorhersehbares Verhalten
Update	Endlosschleife! (Abbrechen mit Strg + C in der Konsole)

Goldene Regel: Die Bedingung muss durch das Update irgendwann **false** werden. Überlege dir das bei jeder Schleife!

8.2 `while` : prüft vor jedem Durchlauf (0 bis n Wiederholungen)

```
while (bedingung) {
    anweisungen;
}
```

Muster: Zählschleife

```
int count = 0;
while (count < 5) {
    System.out.println("*");
    count = count + 1;
}
System.out.println("done!");
```

Ausgabe: fünf Zeilen `*`, dann `done!`. *Frage von der Folie: Was passiert, wenn du zusätzlich `count` aus gibst?* Du siehst `0 1 2 3 4`. Nach der Schleife ist `count` gleich 5.

Muster: Summieren (Akkumulator)

```
Scanner scan = new Scanner(System.in);
int sum = 0; // Akkumulator mit 0 starten
int count = 0;
while (count < 5) {
    int value = scan.nextInt();
    sum = sum + value; // aufaddieren
    count = count + 1;
}
System.out.println("sum is " + sum);
```

8.3 `for`: die kompakte Zählschleife

Eine `for`-Schleife tut **genau dasselbe** wie eine `while`-Schleife, nur stehen alle drei Zutaten in einer Zeile:

```
for (initialisierung; bedingung; update) {
    anweisungen;
}

for (int i = 0; i < 5; i++) {
    System.out.println("*");
}
```

Ablauf: **init** → Bedingung prüfen → (true) → Rumpf → **update** → Bedingung prüfen → ... → (false) → Ende.

Ziel	Schleifenkopf
5 × wiederholen	<code>for (int i = 0; i < 5; i++)</code>
1 bis n	<code>for (int i = 1; i <= n; i++)</code>
n runter bis 0	<code>for (int i = n; i >= 0; i--)</code>
nur gerade Zahlen bis 20	<code>for (int i = 0; i <= 20; i += 2)</code>

i Eine Variable, die im `for`-Kopf deklariert wird (`int i`), existiert **nur innerhalb** der Schleife.

Wann `for`, **wann** `while`? - Anzahl der Durchläufe **bekannt** (zählen): `for` - Anzahl **unbekannt** („solange ...“, „bis der User ...“): `while`

8.4 `do-while`: prüft nach jedem Durchlauf (mindestens 1 Durchlauf)

```
do {
    anweisungen;
} while (bedingung);    // ⚠ Semikolon am Ende!
```

Perfekt für Eingabeprüfung, weil man erst fragen muss, bevor man prüfen kann:

```
Scanner scan = new Scanner(System.in);
int value;
do {
    System.out.println("Enter a positive value:");
    value = scan.nextInt();
} while (value <= 0);    // wiederholen, solange die Eingabe UNGÜLTIG ist
```

Perfekt für Menüs:

```

Scanner scanner = new Scanner(System.in);
int selection;
do {
    System.out.println("ABC Trading Co.");
    System.out.println("-----");
    System.out.println("1 - sales");
    System.out.println("2 - stock");
    System.out.println("3 - admin");
    System.out.print("Select (0 to exit): ");
    selection = scanner.nextInt();

    if (selection == 1) {
        System.out.println("...sales things...");
    } else if (selection == 2) {
        System.out.println("...stock things...");
    } else if (selection == 3) {
        System.out.println("...admin things...");
    } else if (selection != 0) {
        System.out.println("invalid selection");
    }
} while (selection != 0);
System.out.println("goodbye");

```

8.5 break und continue

- `break` beendet die Schleife **sofort**. Es geht mit der ersten Anweisung **nach** der Schleife weiter.
- `continue` bricht nur den **aktuellen Durchlauf** ab und springt zurück zur Bedingung (bei `for`: zuerst zum Update).

```

// break: Endlosschleife mit Ausstiegspunkt
int sum = 0;
while (true) {
    System.out.println("Type in a number, enter -1 to exit:");
    int number = scan.nextInt();
    if (number == -1) {
        break; // raus aus der Schleife
    }
    sum = sum + number;
}
System.out.println("sum is " + sum);

```

```
// continue: ungültige Eingaben überspringen
while (true) {
    System.out.println("Insert positive integers");
    int number = scanner.nextInt();
    if (number <= 0) {
        System.out.println("Unfit number! Try again.");
        continue; // zurück an den Anfang
    }
    System.out.println("Your input was " + number);
}
```

8.6 Verschachtelte Schleifen

Eine Schleife in einer Schleife: Die **innere** läuft bei **jedem** Durchlauf der äußeren komplett durch. Typisch für Muster, Tabellen und (später) 2D-Arrays.

```
int height = 5;
for (int i = 1; i <= height; i++) { // äußere Schleife: Zeile i
    for (int j = 0; j < i; j++) { // innere Schleife: i Sterne
        System.out.print("*");
    }
    System.out.println(); // nach jeder Zeile: Umbruch
}
```

```
*
**
***
****
*****
```

i Auf der Folie beginnt die äußere Schleife bei `i = 0` mit `i <= height`. Das erzeugt zuerst eine **leere** Zeile und dann die 5 Sternzeilen. Mit `i = 1` (wie oben) fällt die Leerzeile weg.

! `break` und `continue` wirken nur auf die **innerste** Schleife, in der sie stehen.

8.7 Terminiert die Schleife? (Folienaufgabe)

```
int count = 0;
while (count < 5) {
    System.out.println("*");
    count = count - 5;    // count: 0, -5, -10, ... bleibt immer < 5
}
System.out.println("done");
```

➡ **Endlosschleife**, `done` wird nie ausgegeben. Das Update bewegt `count` in die falsche Richtung.

```
int i = 1;
while (i != 50) {
    System.out.println(i);
    i = i + 2;    // i: 1, 3, 5, ..., 49, 51, ... trifft 50 NIE
}
System.out.println("done");
```

➡ **Endlosschleife**, weil `i` immer ungerade ist. **Tipp:** Verwende `<` statt `!=`, also `while (i < 50)`. Das ist robuster.

8.8 Tracing: Code von Hand durchspielen

Ein wichtiges Werkzeug (kommt auch in Prüfungen vor): Schreib eine Tabelle mit allen Variablen und spiel den Code Zeile für Zeile durch.

```
int sum = 0;
for (int i = 1; i <= 4; i++) {
    sum += i;
}
```

Durchlauf	i	Bedingung <code>i <= 4</code>	sum danach
Start	1	true	1
2	2	true	3
3	3	true	6
4	4	true	10
-	5	false → Ende	10

✓ Selbsttest 8

1. Wie oft läuft `for (int i = 3; i < 10; i += 3)` ? Welche Werte hat `i` ?
2. Was ist der Hauptunterschied zwischen `while` und `do-while` ?
3. Schreibe eine Schleife, die `10 8 6 4 2 0` ausgibt.

▼ Lösung

1. 3-mal: `i` = 3, 6, 9 (12 ist nicht mehr < 10)
2. `while` prüft vorher (0 oder mehr Durchläufe), `do-while` prüft nachher (mindestens 1 Durchlauf).
3.

```
for (int i = 10; i >= 0; i -= 2) { System.out.print(i + " "); }
```

9. Methoden

9.1 Was ist eine Methode?

Eine **Methode** ist ein **benanntes Unterprogramm**, das du **aufrufen** kannst. Du kennst schon welche: `System.out.println(...)`, `scanner.nextInt()`, `Math.sqrt(...)`. Die hat jemand anderes für dich geschrieben. Jetzt schreibst du eigene.

In anderen Sprachen heißen sie **Funktionen**. In Java heißen sie **Methoden**, weil sie immer in einer Klasse stehen.

Wozu? - **wiederverwendbar**: DRY („Don't Repeat Yourself“) - **organisiert**: Code in klare Teile gliedern - **wartbar**: Fehler nur an **einer** Stelle beheben - **aussagekräftig**:

`isEven(n)` liest sich besser als `n % 2 == 0` - **Abstraktion**: Beim Aufrufen musst du nur wissen, *was* die Methode tut, nicht *wie*.

Wie in der Mathematik: `z = f(a, b, c)`. Eine Methode hat **0 oder mehr Eingaben** (Parameter) und **0 oder 1 Ausgabe** (Rückgabewert).

Ablauf beim Aufruf: Das Programm springt in die Methode, führt sie aus, springt zurück an die Aufrufstelle und hat dort den Rückgabewert.

```
double area = Math.PI * Math.pow(radius, 2);  
//           └─ Sprung zu pow mit (5.0, 2), Rückkehr mit 25.0
```

9.2 Aufbau (Deklaration / Signatur)

```
modifizier rückgabetyt methodenName(parameterliste) {  
    anweisungen;  
}  
  
public static int sum(int a, int b) {  
    return a + b;  
}
```

Teil	Bedeutung
<code>public static</code>	Modifier (siehe 9.6). Für Block 1 bis 4 schreibst du immer <code>public static</code> .
<code>int</code>	Rückgabebetyp : welchen Typ das Ergebnis hat. <code>void</code> = kein Ergebnis.
<code>sum</code>	Name: lowerCamelCase, am besten ein Verb (<code>printX</code> , <code>calculateY</code> , <code>isZ</code>)
<code>(int a, int b)</code>	Parameter : kommasetrennte Liste von <code>Typ Name</code> -Paaren, darf leer sein: <code>()</code>
<code>return a + b;</code>	liefert das Ergebnis an den Aufrufer zurück

9.3 Wo stehen Methoden?

In der Klasse, aber außerhalb jeder anderen Methode (also nicht in `main` hinein!). Die Reihenfolge der Methoden in der Klasse ist egal.

```
public class MyClass {

    public static void myMethod1(String s) {    // void: gibt nichts zurück
        System.out.println(s);
    }

    public static int myMethod2(int i) {        // int: gibt eine Zahl zurück
        return i * 2 + 1;
    }

    public static void main(String[] args) {
        myMethod1("Hello");                    // Aufruf ohne Rückgabewert
        int j = myMethod2(5);                  // Aufruf, Ergebnis (11) wird
        System.out.println("j=" + j);          // j=11
    }
}
```

Aufruf aus einer **anderen Klasse**: `KlassenName.methodenName(...)`, z. B. `Program.showErrorMsg();` (genau wie `Math.sqrt(...)`).

9.4 Parameter

Parameter sind „Platzhalter“ für Werte, die beim Aufruf übergeben werden. Die übergebenen Werte heißen **Argumente**.

```
public static void showErrorMsg(String msg) {           // msg = Parameter
    System.out.println("Error: " + msg);
}

showErrorMsg("an error occurred");                     // Argument
showErrorMsg("number is bigger than 10");
```

Mehrere Parameter werden **in der Reihenfolge** zugeordnet:

```
public static void printDiff(int x, int y) {
    System.out.println("the diff is " + (x - y));
}

printDiff(3, 4);    // x=3, y=4  -> the diff is -1
printDiff(10, 2);   // x=10, y=2 -> the diff is 8
printDiff(2, 10);   // x=2, y=10 -> the diff is -8
```

⚠ Anzahl, Reihenfolge und Typen der Argumente müssen zur Parameterliste passen.
`printDiff(3)` oder `printDiff("a", 4)` sind Compilerfehler.

9.5 Pass-by-Value und Scope (Gültigkeitsbereich)

Pass-by-Value: Beim Aufruf werden die Werte **kopiert**. Änderst du den Parameter in der Methode, ändert sich die Variable des Aufrufers **nicht**.

```

public static void main(String[] args) {
    int min = 5;
    int max = 10;
    printNumbers(min, max);    // gibt 5 6 7 8 9 aus
    System.out.println();
    min = 8;
    printNumbers(min, max);    // gibt 8 9 aus
}

public static void printNumbers(int min, int max) {
    while (min < max) {
        System.out.println(min);
        min++;                // ändert nur die KOPIE, main.min bleibt unverändert
    }
}

```

Scope: Eine Variable existiert **nur in dem Block** `{ }`, in dem sie **deklariert wurde**. Verschiedene Methoden können deshalb Variablen mit **gleichem Namen** haben, die nichts miteinander zu tun haben (wie `min` oben).

```

public static void main(String[] args) {
    int x = 10;
    greet();
    System.out.println("x in main: " + x);    // ✓
}

public static void greet() {
    int y = 5;
    System.out.println("y in greet: " + y);    // ✓
    System.out.println(x);                    // ✗ cannot find symbol: x gibt es nicht
}

```

➡ Wenn eine Methode einen Wert braucht, **übergib ihn als Parameter**. Wenn der Aufrufer ein Ergebnis braucht, **gib es mit** `return` **zurück**.

9.6 Modifier

Modifier	Wirkung
<code>public</code>	von überall aufrufbar
<code>protected</code>	aus demselben Package und aus Unterklassen (Vererbung, Block 8)
<i>(keiner)</i>	nur aus demselben Package
<code>private</code>	nur innerhalb der eigenen Klasse
<code>static</code>	gehört zur Klasse , nicht zu einem Objekt. Man kann sie ohne Objekt aufrufen, wie <code>main</code> .
<code>final</code>	bei Variablen: konstant, nicht mehr änderbar

(Objekte und damit Methoden ohne `static` kommen in Block 6.)

9.7 `return`

```
return wert;
```

- Der Typ von `wert` muss zum **Rückgabebetyp** passen.
- `return` **beendet die Methode sofort**. Code danach im selben Zweig wird nie ausgeführt.
- Mehrere `return`s in verschiedenen Zweigen sind erlaubt, aber **jeder mögliche Weg** durch eine Nicht-`void`-Methode muss mit `return` enden. Sonst meldet der Compiler `missing return statement`.
- In `void`-Methoden darfst du `return;` (ohne Wert) zum vorzeitigen Beenden verwenden.

```
public static double hypotenuse(double side1, double side2) {  
    double side3; // lokale Variable  
    side3 = Math.sqrt(side1 * side1 + side2 * side2);  
    return side3;  
}  
  
double z = hypotenuse(3, 4); // 5.0
```

```
// lang:
public static boolean isEven(int number) {
    if (number % 2 == 0) {
        return true;
    } else {
        return false;
    }
}

// kurz und elegant: der Vergleich IST schon ein boolean
public static boolean isEven(int number) {
    return number % 2 == 0;
}
```

9.8 Methoden kombinieren

Methoden können andere Methoden aufrufen, und das Ergebnis einer Methode kann direkt als Argument dienen:

```
public static int getNumberFromUser() {
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter a number:");
    return scanner.nextInt();
}

public static boolean isEven(int number) {
    return number % 2 == 0;
}

public static void main(String[] args) {
    boolean isNumberEven = isEven(getNumberFromUser()); // erst innen, dann
    if (isNumberEven) {
        System.out.println("the given number is even.");
    } else {
        System.out.println("the given number is odd.");
    }
}
```

9.9 `void` vs. Rückgabewert: die häufigste Verwirrung

	<code>void</code> -Methode	Methode mit Rückgabewert
Zweck	tut etwas (z. B. ausgeben)	berechnet etwas
Aufruf	als eigene Anweisung: <code>println();</code>	Wert wird verwendet: <code>int s = sum(2, 3);</code>
Falsch	<code>int x = println();</code> ❌	<code>sum(2, 3);</code> ist erlaubt, aber sinnlos, weil das Ergebnis verloren geht

⚠️ `println` in einer Methode ist nicht dasselbe wie `return` ! Wenn eine Aufgabe sagt „die Methode soll ... zurückgeben“, dann brauchst du `return`, nicht `System.out.println`.

9.10 Begriffsübersicht

Begriff	Bedeutung	Beispiel
Deklaration / Signatur / Header	Definition mit Modifier, Rückgabotyp, Name, Parametern	<code>public static int sum(int a, int b)</code>
Aufruf (call / invoke)	Methode ausführen	<code>sum(2, 3)</code>
Parameter	Variable in der Deklaration	<code>int a</code>
Argument	konkreter Wert beim Aufruf	<code>2</code>
Rückgabewert	Ergebnis, das zurückgeht	<code>return a + b;</code>
Rückgabotyp	Typ des Ergebnisses	<code>int</code> , <code>void</code>
Access Modifier	Sichtbarkeit	<code>public</code> , <code>private</code>
static	gehört zur Klasse	<code>public static void main(...)</code>

✅ Selbsttest 9

- Schreibe die Signatur einer Methode, die zwei `double`-Werte bekommt und deren Durchschnitt liefert.
- Warum kompiliert das nicht? `java public static int sign(int n) { if (n > 0) return 1; else if (n < 0) return -1; }`

3. Was gibt dieses Programm aus? `java public static void main(String[] args) { int a = 3; doubleIt(a); System.out.println(a); } public static void doubleIt(int a) { a = a * 2; }`

▼ Lösung

1. `public static double average(double a, double b)` mit dem Rumpf `return (a + b) / 2;`
2. Für `n == 0` gibt es keinen `return`: `missing return statement`. Lösung: am Ende `return 0;` ergänzen.
3. 3. Pass-by-Value: Nur die Kopie wird verdoppelt. Richtig wäre `int doubleIt(int a) { return a * 2; }` und in `main` dann `a = doubleIt(a);`.

10. Fehler lesen & Debugging

Fehler sind **normal**, auch für Profis. Wichtig ist, sie lesen zu können.

10.1 Drei Arten von Fehlern

Art	Wann?	Beispiel
Compilerfehler (Syntax/Typ)	beim <code>javac</code> , das Programm startet gar nicht	fehlendes <code>;</code> , falscher Typ, Tippfehler
Laufzeitfehler (Exception)	beim Ausführen, das Programm stürzt ab	Division durch 0, <code>parseInt("abc")</code>
Logikfehler (Bug)	das Programm läuft, macht aber das Falsche	<code>7 / 2</code> statt <code>7 / 2.0</code> , <code><</code> statt <code><=</code>

10.2 Eine Fehlermeldung lesen

```
App.java:7: error: cannot find symbol
    system.out.println("Hi");
    ^
symbol:   variable system
location: class App
```

➔ **Datei und Zeile** (`App.java:7`), **was** (`cannot find symbol`), **wo** (`^`), **welches Symbol** (`system`). Hier fehlt das große **S** in `System`.

💡 **Tipp:** Immer den **ersten** Fehler zuerst beheben. Folgefehler verschwinden oft von selbst.

10.3 Die häufigsten Meldungen

Meldung	Typische Ursache
<code>';' expected</code>	Semikolon vergessen (oft in der Zeile davor!)
<code>cannot find symbol</code>	Typfehler, falsche Groß-/Kleinschreibung, Variable nicht deklariert oder außerhalb des Scopes, <code>import</code> fehlt (z. B. bei <code>Scanner</code>)
<code>incompatible types: possible lossy conversion from double to int</code>	Kommazahl in <code>int</code> -Variable, also Cast nötig oder Typ ändern
<code>incompatible types: String cannot be converted to int</code>	Text in Zahl-Variable, also <code>Integer.parseInt(...)</code> verwenden
<code>variable x might not have been initialized</code>	Variable vor dem Lesen nie belegt
<code>missing return statement</code>	nicht jeder Weg endet mit <code>return</code>
<code>reached end of file while parsing</code>	eine <code>}</code> fehlt
<code>class X is public, should be declared in a file named X.java</code>	Dateiname $\neq$ Klassenname
<code>unreachable statement</code>	Code nach <code>return</code> oder <code>break</code>
<code>ArithmeticException: / by zero</code>	Laufzeit: int-Division durch 0
<code>InputMismatchException</code>	Laufzeit: Scanner erwartet Zahl, bekommt Text
<code>NumberFormatException</code>	Laufzeit: <code>parseInt</code> mit ungültigem Text

10.4 Debugging-Strategien

1. **Kontrollausgaben:** `System.out.println("DEBUG i=" + i + " sum=" + sum);` an verdächtigen Stellen einfügen.
2. **Tracing-Tabelle** von Hand (Kapitel 8.8).
3. **Debugger in VS Code:** Klick links neben die Zeilennummer setzt einen roten **Breakpoint**. Dann **Debug** statt **Run** starten und mit *Step Over* Zeile für Zeile durchgehen, dabei links die Variablenwerte beobachten.
4. **Grenzfälle testen:** 0, 1, negative Zahlen, größter Wert, leere Eingabe.
5. **Klein anfangen:** nach jedem kleinen Schritt kompilieren und testen, nicht 50 Zeilen auf einmal schreiben.

11. Übungsaufgaben mit Lösungen

Alle Lösungen sind **getestet** (Java 21) und stehen vollständig im Anhang. Ausführen: in den Ordner wechseln, dann `javac Datei.java` und `java Datei`.

Vorgehen bei jeder Aufgabe (Programmier-Rezept):

1. Aufgabe verstehen: Was ist **Eingabe**, was ist **Ausgabe**? Ein Beispiel von Hand durchrechnen.
2. Schritte als **Pseudocode/Kommentare** notieren.
3. Nötige **Variablen und Typen** festlegen.
4. Code Schritt für Schritt schreiben, **nach jedem Schritt kompilieren**.
5. Mit normalen Werten **und Grenzfällen** testen.

Aufgabe 1: Notenrechner (Recap 02, Folie 1 von 03)

Begrüßung; Name und Punkte (0 bis 40) einlesen; gerade/ungerade prüfen; Prozent berechnen; Rückmeldung: $\geq 90 \rightarrow$ „Excellent“, $\geq 75 \rightarrow$ „Good job“, $\geq 60 \rightarrow$ „Passed“, sonst „Needs improvement“.

▼ Lösung

```
import java.util.Scanner;

public class GradeCheck {
    public static void main(String[] args) {
        final int MAX_POINTS = 40;
        Scanner scanner = new Scanner(System.in);

        System.out.println("Welcome to the grade checker!");

        System.out.print("Your name: ");
        String name = scanner.nextLine(); // Name ZUERST, siehe S
        System.out.print("Your points (0-" + MAX_POINTS + "): ");
        int points = scanner.nextInt();

        if (points % 2 == 0) {
            System.out.println(points + " is even.");
        } else {
            System.out.println(points + " is odd.");
        }

        double percentage = points * 100.0 / MAX_POINTS; // 100.0 gegen
        System.out.println(name + ", you reached " + percentage + " %.");

        if (percentage >= 90) {
            System.out.println("Excellent");
        } else if (percentage >= 75) {
            System.out.println("Good job");
        } else if (percentage >= 60) {
            System.out.println("Passed");
        } else {
            System.out.println("Needs improvement");
        }
    }
}
```

Test: Anna, 31 Punkte ergibt 31 is odd. , 77.5 % , Good job . Datei:
beispiele/GradeCheck.java

Aufgabe 2: Countdown

Startzahl einlesen und bis 0 herunterzählen. Eingabe 5 ergibt `5 4 3 2 1 0`.

▼ Lösung

```
Scanner scanner = new Scanner(System.in);
System.out.print("Start number: ");
int start = scanner.nextInt();

for (int i = start; i >= 0; i--) {
    System.out.print(i + " ");
}
System.out.println();
```

Datei: `beispiele/Countdown.java`

Aufgabe 3: Fakultät

n einlesen und n! berechnen. Eingabe 4 ergibt `4! = 24`.

▼ Lösung

```
Scanner scanner = new Scanner(System.in);
System.out.print("n: ");
int n = scanner.nextInt();

long result = 1; // bei Produkten mit 1 starten, nicht
for (int i = 2; i <= n; i++) {
    result = result * i;
}
System.out.println(n + "! = " + result);
```

💡 `long`, weil schon 13! nicht mehr in einen `int` passt. `0!` ist 1 und funktioniert automatisch, weil die Schleife dann gar nicht läuft.

Datei: `beispiele/Factorial.java`

Aufgabe 4: Passwort mit 3 Versuchen

Maximal 3 Versuche. Bei richtigem Passwort „Access granted“, sonst „Access denied“.

▼ Lösung

```
final String PASSWORD = "java21";
final int MAX_ATTEMPTS = 3;
Scanner scanner = new Scanner(System.in);

boolean granted = false;
int attempts = 0;
while (attempts < MAX_ATTEMPTS && !granted) {
    System.out.print("Password: ");
    String input = scanner.nextLine();
    attempts++;
    if (input.equals(PASSWORD)) {           // equals, NICHT ==
        granted = true;
    } else if (attempts < MAX_ATTEMPTS) {
        System.out.println("Wrong, " + (MAX_ATTEMPTS - attempts) + " attem
    }
}

if (granted) {
    System.out.println("Access granted");
} else {
    System.out.println("Access denied");
}
```

💡 Das **Flag-Muster**: Eine `boolean`-Variable (`granted`) merkt sich, ob etwas passiert ist, und steuert die Schleife mit.

Datei: `beispiele/PasswordRetry.java`

Aufgabe 5: Umgekehrte Treppe

n einlesen, Treppe aus `*` mit n Zeilen, die längste zuerst.

▼ Lösung

```
Scanner scanner = new Scanner(System.in);
System.out.print("n: ");
int n = scanner.nextInt();

for (int row = n; row >= 1; row--) {           // Zeilen: n, n-1, ..., 1
    for (int star = 0; star < row; star++) {   // so viele Sterne wie die Z
        System.out.print("*");
    }
    System.out.println();
}
```

Datei: `beispiele/ReversedStairs.java`

Aufgabe 6: Ziffern zählen

Positive Zahl einlesen und ihre Stellen zählen. Eingabe 12345 ergibt `5 digits`.

▼ Lösung

```
Scanner scanner = new Scanner(System.in);
System.out.print("Positive integer: ");
int number = scanner.nextInt();

int digits = 0;
do {
    number = number / 10; // letzte Ziffer abschneiden: 12345 → 1234 → 1
    digits++;
} while (number > 0);

System.out.println(digits + " digits");
```



`do-while` statt `while`, damit auch die Eingabe `0` korrekt **1** Stelle ergibt.

Datei: `beispiele/CountDigits.java`

Aufgabe 7: `printUntilNumber` (Methoden-Übung 1)

`public static void printUntilNumber(int number)` gibt die Zahlen von 1 bis `number` aus.

▼ Lösung

```
public static void printUntilNumber(int number) {
    for (int i = 1; i <= number; i++) {
        System.out.println(i);
    }
}

public static void main(String[] args) {
    printUntilNumber(5);    // 1 2 3 4 5 (untereinander)
    printUntilNumber(2);    // 1 2
}
```

Aufgabe 8: `sumOfNumbers` (Methoden-Übung 2)

Die Methode bekommt Start- und Endwert und **gibt** die Summe aller Zahlen dazwischen (inklusive) **zurück**. `sumOfNumbers(2, 4)` ergibt 9.

▼ Lösung

```
public static int sumOfNumbers(int start, int end) {
    int sum = 0;
    for (int i = start; i <= end; i++) {
        sum += i;
    }
    return sum;                // zurückgeben, nicht ausgeben!
}

public static void main(String[] args) {
    int sum = sumOfNumbers(2, 4);
    System.out.println(sum);    // 9
    System.out.println(sumOfNumbers(1, 200)); // 20100, Gauß lässt grüße
}
```

Datei mit Aufgabe 7 und 8: `beispiele/MethodExercises.java`

Zusatzaufgaben zum Weiterüben (ohne vorgegebene Lösung)

1. **FizzBuzz:** Gib die Zahlen 1 bis 100 aus. Bei Vielfachen von 3 gib „Fizz“ aus, bei Vielfachen von 5 „Buzz“, bei beidem „FizzBuzz“. (*Achtung: Reihenfolge der Bedingungen!*)
 2. **Schaltjahr-Methode:** `public static boolean isLeapYear(int year)`. Teste 1900 (nein), 2000 (ja), 2024 (ja).
 3. **Zahlenraten:** Das Programm „denkt“ sich eine Zahl (`int secret = 42;`). Der User rät, bis er richtig liegt, und bekommt die Hinweise „zu groß“ oder „zu klein“. Am Ende wird die Anzahl der Versuche ausgegeben.
 4. **Taschenrechner-Menü:** Kombiniere `do-while` -Menü, `switch` für `+` `-` `*` `/` und je eine Methode pro Rechenart. Fange die Division durch 0 ab.
 5. **Quersumme:** `public static int digitSum(int n)`, z. B. 1234 ergibt 10. (*Tipp:* `% 10` und `/ 10`)
 6. **Primzahl:** `public static boolean isPrime(int n)`. Prüfe mit einer Schleife, ob ein Teiler zwischen 2 und $n-1$ existiert.
 7. **Multiplikationstabelle:** Gib mit zwei verschachtelten Schleifen das kleine Einmaleins (1 bis 10) als Tabelle aus. (*Tipp:* `System.out.print(i * j + "\t");`)
-

12. Spickzettel (Cheat Sheet)

```
import java.util.Scanner;                                // für Eingabe, ganz oben

public class Programm {                                  // Klassenname == Dateiname

    // ----- eigene Methode -----
    public static int square(int n) {                    // Rückgabotyp int
        return n * n;
    }

    public static void greet(String name) {              // void: kein Rückgabewert
        System.out.println("Hi " + name);
    }

    // ----- Einstiegspunkt -----
    public static void main(String[] args) {
        // Variablen und Konstanten
        int count = 0;
        double price = 9.99;
        boolean done = false;
        char grade = 'A';
        String name = "Anna";
        final int MAX = 10;

        // Eingabe
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        sc.nextLine();                                    // Enter nach nextInt entsor
        String line = sc.nextLine();

        // Ausgabe
        System.out.println("n = " + n);
        System.out.print("ohne Umbruch ");

        // Verzweigung
        if (n > 0 && n < MAX) {
            // ...
        } else if (n == 0) {
            // ...
        } else {
            // ...
        }

        switch (grade) {
```

```

        case 'A':
            System.out.println("top");
            break;
        default:
            System.out.println("ok");
    }

    // Schleifen
    for (int i = 0; i < 5; i++) { /* 5-mal */ }
    while (!done) { done = true; }
    do { n--; } while (n > 0);

    // Methoden aufrufen
    int q = square(4); // 16
    greet(name);

    // Strings vergleichen
    if (line.equals("yes")) { /* ... */ }

    // Umwandlungen
    int fromText = Integer.parseInt("42");
    int cut = (int) 3.9; // 3
    double avg = (3 + 4) / 2.0; // 3.5
}

```

Die 12 wichtigsten Merksätze

1. **Klassenname = Dateiname**, Java ist **case-sensitive**.
2. Jede Anweisung endet mit `;`, jede `{` braucht ihre `}`.
3. `int / int = int`. Für Kommaergebnisse muss eine Seite `double` sein (`2.0`).
4. `%` liefert den **Rest**: `n % 2 == 0` bedeutet gerade.
5. `=` weist zu, `==` vergleicht.
6. **Strings immer mit** `.equals()` vergleichen.
7. `if / else` **immer mit** `{ }`, kein `;` nach `if (...)`.
8. In `switch` das `break` nicht vergessen (Fall-through!).
9. Jede Schleife braucht **Initialisierung, Bedingung, Update**, sonst droht eine Endlosschleife.
10. `for` zum **Zählen**, `while` für „solange“, `do-while` für „mindestens einmal“ (Eingabepfung, Menüs).
11. Methoden: `return` liefert zurück, `println` gibt nur aus. Parameter sind **Kopien** (Pass-by-Value).
12. Variablen leben nur in **ihrem Block** (Scope).

Anhang: Alle Beispielprogramme

Alle Programme wurden mit Java 21 kompiliert und getestet. Ausführen: `javac Datei.java`, danach `java Datei`.

HelloWorld.java

```
/**
 * Das kleinste lauffähige Java-Programm.
 * Kompilieren: javac HelloWorld.java
 * Ausführen:   java HelloWorld
 */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

AreaCircum.java

```
import java.util.Scanner;

/**
 * AreaCircum - berechnet Fläche und Umfang eines Kreises aus dem Radius.
 * (Komplettbeispiel aus Foliensatz 02)
 */
public class AreaCircum {
    public static void main(String[] args) {
        // Konstanten
        final double PI = 3.142;

        // Variablen
        int radius;
        double area;
        double circumference;
        Scanner scan = new Scanner(System.in);

        // 1. Begrüßung ausgeben
        System.out.println("Welcome to the Area Circumference Calculator");
        // 2. Radius vom Benutzer abfragen
        System.out.println("Please enter the radius: ");
        radius = scan.nextInt();
        // 3. Fläche = pi * r * r
        area = PI * radius * radius;
        // 4. Umfang = 2 * pi * r
        circumference = 2 * PI * radius;
        // 5. Ergebnisse ausgeben
        System.out.println("The area of a circle of radius: " + radius + " is " + area);
        System.out.println("and its circumference is: " + circumference);
    }
}
```

GradeCheck.java

```
import java.util.Scanner;

/**
 * Recap-Übung (Foliensatz 03, Folie 1):
 * Name und Punkte (0-40) einlesen, gerade/ungerade prüfen,
 * Prozent berechnen und eine Rückmeldung ausgeben.
 */
public class GradeCheck {
    public static void main(String[] args) {
        final int MAX_POINTS = 40;
        Scanner scanner = new Scanner(System.in);

        // 1. Begrüßung
        System.out.println("Welcome to the grade checker!");

        // 2. Name und Punkte einlesen
        System.out.print("Your name: ");
        String name = scanner.nextLine();
        System.out.print("Your points (0-" + MAX_POINTS + "): ");
        int points = scanner.nextInt();

        // 3. Gerade oder ungerade?
        if (points % 2 == 0) {
            System.out.println(points + " is even.");
        } else {
            System.out.println(points + " is odd.");
        }

        // 4. Prozent berechnen - 100.0 (double!), sonst Ganzzahldivision
        double percentage = points * 100.0 / MAX_POINTS;
        System.out.println(name + ", you reached " + percentage + " %.");

        // 5. Rückmeldung - von der höchsten Grenze abwärts prüfen
        if (percentage >= 90) {
            System.out.println("Excellent");
        } else if (percentage >= 75) {
            System.out.println("Good job");
        } else if (percentage >= 60) {
            System.out.println("Passed");
        } else {
            System.out.println("Needs improvement");
        }
    }
}
```

```
}  
}
```

Countdown.java

```
import java.util.Scanner;  
  
/** Übung "Countdown": Input 5 -> Output 5 4 3 2 1 0 */  
public class Countdown {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("Start number: ");  
        int start = scanner.nextInt();  
  
        for (int i = start; i >= 0; i--) {  
            System.out.print(i + " ");  
        }  
        System.out.println();  
    }  
}
```

Factorial.java

```
import java.util.Scanner;  
  
/** Übung "Factorial Calculator": Input 4 -> 4! = 24 */  
public class Factorial {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
        System.out.print("n: ");  
        int n = scanner.nextInt();  
  
        long result = 1; // long, weil Fakultäten sehr schnell sehr groß werden  
        for (int i = 2; i <= n; i++) {  
            result = result * i;  
        }  
        System.out.println(n + "! = " + result);  
    }  
}
```

PasswordRetry.java

```
import java.util.Scanner;

/** Übung "Password Retry": maximal 3 Versuche. */
public class PasswordRetry {
    public static void main(String[] args) {
        final String PASSWORD = "java21";
        final int MAX_ATTEMPTS = 3;
        Scanner scanner = new Scanner(System.in);

        boolean granted = false;
        int attempts = 0;
        while (attempts < MAX_ATTEMPTS && !granted) {
            System.out.print("Password: ");
            String input = scanner.nextLine();
            attempts++;
            if (input.equals(PASSWORD)) { // Strings IMMER mit equals vergleichen
                granted = true;
            } else if (attempts < MAX_ATTEMPTS) {
                System.out.println("Wrong, " + (MAX_ATTEMPTS - attempts) + " a
            }
        }

        if (granted) {
            System.out.println("Access granted");
        } else {
            System.out.println("Access denied");
        }
    }
}
```

ReversedStairs.java

```
import java.util.Scanner;

/** Übung "Reversed Stairs": Input 4 -> ****, ***, **, * */
public class ReversedStairs {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("n: ");
        int n = scanner.nextInt();

        for (int row = n; row >= 1; row--) {           // äußere Schleife: Zeilen
            for (int star = 0; star < row; star++) {    // innere Schleife: Sternchen
                System.out.print("*");
            }
            System.out.println();                       // Zeilenumbruch nach jeder Zeile
        }
    }
}
```

CountDigits.java

```
import java.util.Scanner;

/** Übung "Count Digits": Input 12345 -> 5 digits */
public class CountDigits {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Positive integer: ");
        int number = scanner.nextInt();

        int digits = 0;
        do {                                           // do-while, damit auch die Zahl 0 als 1 Stelle zählt
            number = number / 10; // letzte Stelle "abschneiden" (Ganzzahldivision)
            digits++;
        } while (number > 0);

        System.out.println(digits + " digits");
    }
}
```

MethodExercises.java

```
/**
 * Übungen aus Foliensatz 04 (Methoden) plus zwei Zusatzbeispiele.
 */
public class MethodExercises {

    /** Übung 1: gibt die Zahlen von 1 bis number untereinander aus. */
    public static void printUntilNumber(int number) {
        for (int i = 1; i <= number; i++) {
            System.out.println(i);
        }
    }

    /** Übung 2: Summe aller Zahlen von start bis end (beide inklusive). */
    public static int sumOfNumbers(int start, int end) {
        int sum = 0;
        for (int i = start; i <= end; i++) {
            sum += i;
        }
        return sum;
    }

    /** Liefert true, wenn number gerade ist. */
    public static boolean isEven(int number) {
        return number % 2 == 0;
    }

    /** Hypotenuse eines rechtwinkligen Dreiecks (Pythagoras). */
    public static double hypotenuse(double side1, double side2) {
        return Math.sqrt(side1 * side1 + side2 * side2);
    }

    public static void main(String[] args) {
        printUntilNumber(5);
        System.out.println("---");
        printUntilNumber(2);

        System.out.println("sumOfNumbers(2, 4) = " + sumOfNumbers(2, 4));
        System.out.println("sumOfNumbers(1, 200) = " + sumOfNumbers(1, 200));

        System.out.println("isEven(7) = " + isEven(7));
        System.out.println("hypotenuse(3, 4) = " + hypotenuse(3, 4));
    }
}
```

Handshakes.java

```
/**
 * Aufgabe "Handshake Summit of Sillyville" (Foliensatz 01):
 * Wie viele Handschläge gibt es, wenn jede von n Personen jeder anderen
 * genau einmal die Hand gibt?
 */
public class Handshakes {

    /** Variante 1: Schleife - Person 1 gibt n-1 Hände, Person 2 noch n-2 neue
    public static int countWithLoop(int people) {
        int total = 0;
        for (int newHands = people - 1; newHands >= 1; newHands--) {
            total += newHands;
        }
        return total;
    }

    /** Variante 2: Formel (Abstraktion) -  $n * (n - 1) / 2$  */
    public static int countWithFormula(int people) {
        return people * (people - 1) / 2;
    }

    public static void main(String[] args) {
        int citizens = 50;
        System.out.println("Loop: " + countWithLoop(citizens));
        System.out.println("Formula: " + countWithFormula(citizens));
    }
}
```

Pitfalls.java

```
/**
 * Typische Stolperfallen zum Selbst-Ausprobieren.
 */
public class Pitfalls {
    public static void main(String[] args) {
        // 1. Ganzzahldivision
        System.out.println(7 / 2);           // 3   (int / int -> int, Rest wird
        System.out.println(7 / 2.0);         // 3.5 (sobald ein double beteiligt
        System.out.println(7 % 2);           // 1   (Rest)

        int z = 4 + 2 / 3 - 1;                // 2/3 ist 0 -> 4 + 0 - 1
        System.out.println("z = " + z);       // 3

        final int TAXRATE = 20;
        int gross = 234;
        double netWrong = gross * (1 - TAXRATE / 100); // 20/100 = 0 -> 234
        double netRight = gross * (1 - TAXRATE / 100.0); // 0.2 -> 234
        System.out.println("net wrong = " + netWrong + ", net right = " + netR

        // 2. Strings mit + : Reihenfolge zählt (von links nach rechts)
        System.out.println("Sum: " + 1 + 2);    // Sum: 12
        System.out.println("Sum: " + (1 + 2));  // Sum: 3
        System.out.println(1 + 2 + " is the sum"); // 3 is the sum

        // 3. Casting
        double d = 9.99;
        int cut = (int) d;                       // Nachkommastellen werden abgeschni
        System.out.println("(int) 9.99 = " + cut);
        long rounded = Math.round(d);            // runden
        System.out.println("Math.round(9.99) = " + rounded);

        // 4. Strings vergleichen
        String a = "hello";
        String b = new String("hello");
        System.out.println("a == b -> " + (a == b)); // false (versc
        System.out.println("a.equals(b) -> " + a.equals(b)); // true (gleich

        // 5. i++ vs ++i
        int i = 5;
        int post = i++; // post = 5, danach i = 6
        int pre = ++i; // i = 7, danach pre = 7
        System.out.println("post=" + post + " pre=" + pre + " i=" + i);

        // 6. Ternärer Operator
```

```
int grade = 55;  
String result = (grade >= 50) ? "Pass" : "Fail";  
System.out.println(result);  
}  
}
```

Erstellt auf Basis der Folien „Programmierung 1 ILV“ (Hochschule Campus Wien), Blöcke 00 bis 04. Wo die Folien kleine Fehler enthalten, ist das im Skript markiert (⚠).